



Intel® Ethernet Controller E810

eSwitch switchdev Mode

Technology and Configuration Guide

Ethernet Products Group (EPG)

August 2021

Revision 1.0
645272-001

Revision History

Revision	Date	Comments
1.0	August 30, 2021	Initial public release.

Contents

1.0	Known Issues - Read First	5
2.0	Introduction	5
3.0	Hardware and Software Requirements	5
4.0	eSwitch Mode (switchdev and Legacy)	6
4.1	Overview	6
4.1.1	Technical Details: eSwitch Legacy Mode	6
4.1.2	Technical Details: eSwitch switchdev Mode	7
4.1.2.1	Default/Exception/Slow Path	9
4.1.2.2	Bridge-Based Slow Data Path	10
4.1.3	Technical Details: switchdev Mode and TC-Flower	10
4.1.3.1	Switchdev Mode TC-Flower Hardware Offloads	10
5.0	eSwitch Configuration	11
5.1	eSwitch Mode Configuration Between Legacy/switchdev	11
5.1.1	Overview	11
5.1.2	Variable Definitions	12
5.1.3	Configuration Steps	12
5.2	Limitations and Troubleshooting	13
5.3	TC-Flower Rule Hardware Offload Configuration	14
Appendix A	Sample Scripts	16
A.1	Script A: eSwitch switchdev Mode with Linux Bridge Configuration	16
A.2	Script B: Switchdev Mode and OVS Configuration	18
Appendix B	Glossary and Acronyms	21

NOTE: ***This page intentionally left blank.***

1.0 Known Issues - Read First

The code contains the following known issues:

- Flow counters or per-flow statistics are not currently supported by the Intel® Ethernet 800 Series (800 Series) for hardware offloaded flows. Therefore, the Open Virtual Switch (OVS) software does not detect hardware offloaded flows as active and deletes hardware offloaded flows after a preset max-idle value (default 10 seconds, though it can be extended to 10 hours). After the OVS max-idle time for each flow, packets are re-passed through the slow path before again being offloaded to hardware until the next timeout value is reached.
- Unable to change the Maximum Transmission Unit (MTU) value on VF Port Representor. An error is thrown when trying to change the MTU value of VF netdev Port Representor.
- Duplicate packets seen after deleting and adding **ovs-ofctl** rules. First add the OVS rules and start the traffic so flows are offloaded. Delete the OVS rules on the software bridge, then configure the OVS rules again and start traffic. Flows are offloaded and traffic passed, but duplicate packets are seen.

2.0 Introduction

The Intel® Ethernet 800 Series is the next generation of Intel Ethernet Controllers and Network Adapters. The 800 Series is designed with an enhanced programmable pipeline, allowing deeper and more diverse protocol header processing. It has capabilities like intelligent offloads to enable high performance, I/O virtualization for max performance in a virtualized server, and Intel® Ethernet Adaptive Virtual Function (Intel® Ethernet AVF) to ease SR-IOV migration to future Intel Ethernet products.

This document describes a new feature introduced in 800 Series to support a new switchdev mode for the controller's embedded switch (eSwitch). This feature allows the adapters to support hardware assist for virtual switch (OVS, Linux Bridge) filters.

This document describes:

- The theory of the eSwitch in both legacy and switchdev modes.
- The advantages of switchdev mode.
- Some limitations with switchdev mode.
- How to configure switchdev mode on 800 Series Network Adapters, including OVS filter configuration.

Note: For configuration details, refer to [Appendix A, "Sample Scripts"](#).

3.0 Hardware and Software Requirements

Following are the hardware and software requirements for eSwitch legacy/switchdev mode support:

- [Intel® Ethernet 800 Series Adapter](#) (which supports switchdev functionality).
- The latest [NVM update package](#) (version 2.50 or later).
- The latest [Linux ice driver](#) (version 1.5.8 or later - out of tree).
- The latest [Linux iavf driver](#) (version 4.0.1 or later).
- Linux kernel-based OS (with kernel version 4.12+ or equivalent) that is supported by 800 Series Software/NVM release (refer to the [Intel® Ethernet Controller E810 Feature Support Matrix](#)).

Did this document help answer your questions?

4.0 eSwitch Mode (switchdev and Legacy)

4.1 Overview

Modern network controllers have a complex embedded switch, referred to as an eSwitch or a Virtual Ethernet Bridge (VEB). This eSwitch is configured and controlled by the Physical Function driver (PF driver or LAN driver) per-port on the Ethernet controller.

Starting with the Linux *ice* PF driver 1.5.8 and NVM 2.50 on 800 Series Ethernet Network Controllers, the eSwitch can be configured per-PF into one of two states via a **devlink** interface:

- eSwitch Legacy mode (default)
- eSwitch switchdev mode

In the default state (referred to as Legacy mode), the PF driver has limited control of the VFs attached to it. SR-IOV-based VMs, or Containers in the default eSwitch legacy mode, bypass the hypervisor and the software virtual switch (OVS or Linux Bridge) completely. By default, only MAC and VLAN filters are added by the PF driver to the VFs at the hardware level. All other configuration is added through software (either through the Linux Bridge or OVS) and does not take advantage of hardware switching capabilities by the Ethernet controller.

In switchdev mode, the PF driver supports a standard Linux kernel abstraction layer (switchdev) to expose the control plane of the Ethernet controller's eSwitch to the software vSwitch (OVS/Linux Bridge). Each port attached to the eSwitch is represented as a Port Representor (PR) netdev that provides a hook to configure and control VM guest or container VF network interfaces. This enables limited vSwitch functionality offload to the Ethernet controller to allow for OVS/Linux Bridge hardware assist support.

[Section 4.1.2](#) explains more about the eSwitch in switchdev mode on an 800 Series Ethernet Network Controller: how to switch between eSwitch modes, how to outline the routing behavior, and how to determine which filters can be offloaded (limited OVS acceleration support).

4.1.1 Technical Details: eSwitch Legacy Mode

Legacy mode is the default state of the Intel® Ethernet Controller E810's eSwitch. This is, how the eSwitch does hardware switching in a traditional network controller with SR-IOV.

In SR-IOV there is a PCIe PF per external LAN port, and there is an eSwitch per {PF: LAN port} pair that connects all the VF Virtual Station Interfaces (VSIs) attached to the external port. Each VF has its own PCI configuration space, so software resources associated for data transfer are directly available to the VF and are isolated from use by the other VFs and the PF. When SR-IOV VFs are assigned to a VM in pass-through mode, the system IOMMU controls and protects memory access. This is the reason why host admin or PF has limited control over VF hardware configuration once the VF is assigned to VM/VNF or container.

The VF is a light-weight PCI function and capable of doing only a limited number of tasks compared to the PF, mostly transmit and receive. However, it is capable of doing this directly to the VF hardware, thus achieving the goal of bypassing the hypervisor for I/O operations.

In this mode, configuration and filters configured through OVS-TC or the Linux Bridge for VFs are configured and used only in software switching (vSwitch) without hardware switching support. MAC and VLAN hardware level filter for the VF are configured automatically by PF driver and only these filters (VLAN/MAC) are configured in the hardware eSwitch for hardware switching.

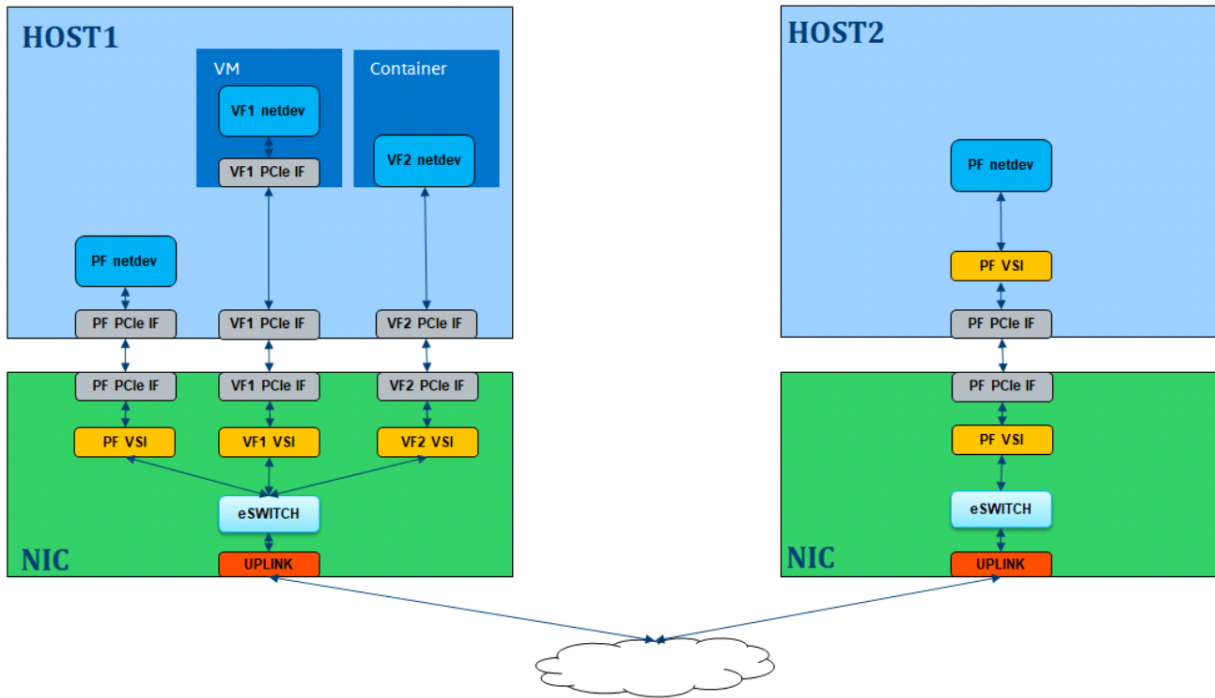


Figure 1. eSwitch in Legacy mode

4.1.2 Technical Details: eSwitch switchdev Mode

In switchdev mode, the 800 Series Controller's PF driver supports a standard Linux kernel abstraction layer (switchdev API) to expose the control plane of the Ethernet controller's eSwitch to the software vSwitch (OVS/Linux Bridge). This switchdev API was originally developed in the Linux Kernels to configure switch hardware ASICs. The switchdev API enables limited vSwitch functionality offload to the Ethernet controller's eSwitch, to allow hardware switch assist for OVS/Linux Bridge.

As detailed in [Section 4.1.1](#), in Legacy mode (without switchdev), only MAC/VLAN are configured and used in VEB (eSwitch) for hardware switching. Based on policies configured by the host admin or SDN controller, flows are configured in the software switch (vSwitch) through a "slow path" mechanism. Flows in this case refer to (filter-match)/action tables for different filter/classifier (L2/L3 and L4 fields). To make use of hardware switching capabilities for vSwitch, it is required for the eSwitch to have same flows similar to vSwitch.

When the PF's eSwitch mode is set as "switchdev", Port Representor netdevs are created for the PF and each VF associated to that eSwitch. The PF netdev is replaced by an UpLink Port Representor (UL_PR) netdev and a VF Port Representor (VF_PR) netdev is created for each VF. PRs are like hardware accelerated networking ports as standard Linux network interfaces. The PF_VSI backs up the PF (UL_PR) netdev and a new Control Plane VSI (CP_VSI) is created, and it backs up all the VF_PR netdevs. PF_PR can be used as two-way communication ports with VFs. VF_PRs are used for control plane communications via OVS control plane.

These PRs enable exposing statistics as well as configuring and monitoring link state, MTU, filters, FDB/VLAN entries, and so on. These PRs plug nicely into existing kernel software switching subsystems (such as TC and OVS), and allow offload of software traffic rules (flows) to the Ethernet controller (hardware).

To offload a data forwarding path (software vSwitch flows) from the kernel to Ethernet controller hardware (eSwitch), the bridge Forwarding Database (FDB) entries are mirrored down to the Ethernet controller. By default, the hardware FDB has entries for {MAC, VLAN, PORT} tuples.

With switchdev mode, the 800 Series Ethernet Network Controller PF driver also supports switch rule programming related to (L3/L4 tuple), hence FDB with L3/L4 tuple can be offloaded. Thus, L2, L3, and L4 tuple (L2/L3/L4 Header fields) can be used for hardware switching.

For example, when a route is added in OVS, a function calls the Intel *switchdev* driver, which then determines whether this route needs to be offloaded to the hardware. Routes that do not involve the eSwitch are typically not offloaded.

Following is a summary on Port Representor capabilities:

- Netdevs for all switching ports created on an eSwitch in switchdev mode:
 - Uplink ports (PF_PR)
 - Virtual ports (VF_PR)
- PR reflects the originating item's link state.
- PR should not be used to control Receive Side Scaling (RSS) input set or Intel® Ethernet Flow Director (Intel® Ethernet FD) settings for the VF.
- Supports default/exception paths.
- When packets are sent from the PR netdev, they are sourced routed directly to the port.
- Scope is limited to single Uplink port in one switching domain/switchdev instance.

Figure 2 shows the configuration of a single port in switchdev mode.

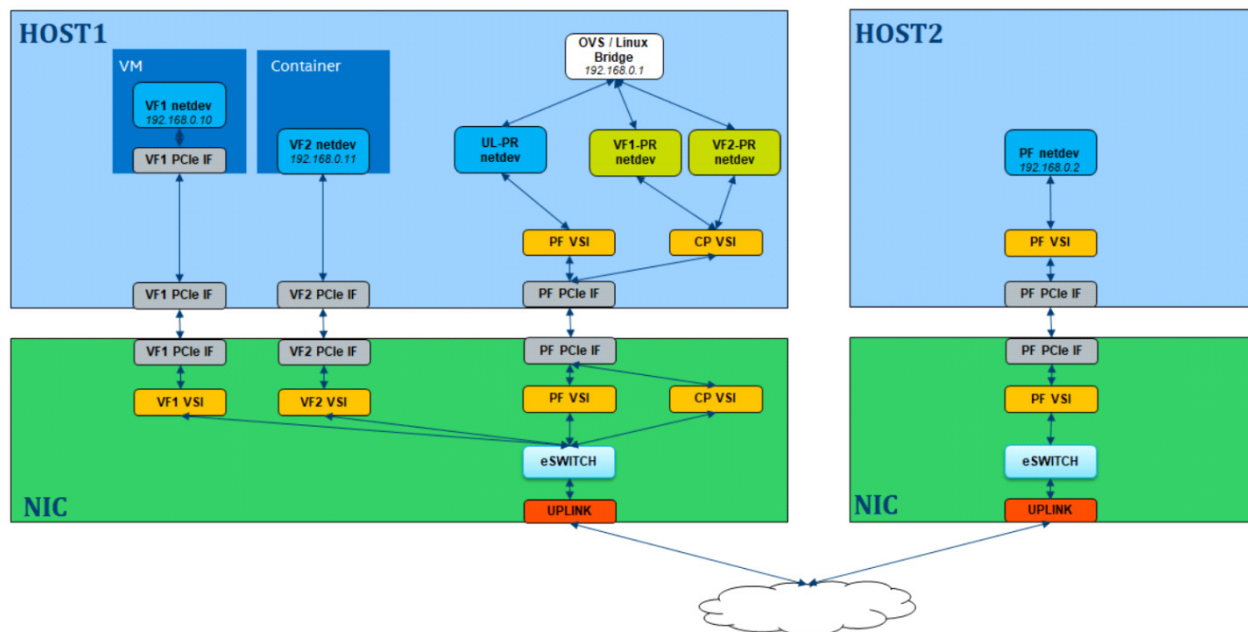


Figure 2. eSwitch in switchdev Mode and PR

VF Port Representor:

Admin has the choice to rename the VF Port Representors like VF_PR, as a VF Port Representor for each VF. Each VF must have a unique Port Representor. This is recommended to use corresponding VF_PRs for any VF-related configuration.

For example, if admin wants to limit a VF's interrupt rate for Rx and Tx ("Bounding interrupt rates using rx-usecs-high"), it applies the command using the VF_PR, as shown here:

```
ethtool -C $<VF_PR> rx-usecs-high $<interrupt rate cap>
```

4.1.2.1 Default/Exception/Slow Path

In switchdev mode, the flow rules are programmed by the control plane. An exception path is enabled via CP_VSI to allow a virtual switch like OVS or Linux bridge to receive any packet that does not match any hardware filter and program the flow rules based on the policy configured by the host admin or SDN controller. The packet is also re-injected to the right port. This path is known as the slow path.

Initially by default, hardware eSwitch has only (MAC/VLAN) entries. Other configuration and filters are not present on the hardware eSwitch. For every first packet received on the external LAN port, mostly there is no matching flow rule in the hardware eSwitch, so the first packet always follows Default or exception path via CP_VSI to reach to software vSwitch (OVS or Linux bridge).

The vSwitch has flow rules configured as per host admin or SDN controller. When matching flow entry is hit in the software switch control plane (OVS/Linux Bridge), pre-defined match action is performed. At the same time, that match/action flow rule is also configured in the hardware eSwitch (classifier engine). This way, a mirror of software control plane/FDB is created in hardware eSwitch.

When second packet hits the LAN Port, there is a matching entry in the hardware eSwitch's control plane/FDB, and hardware performs the required match action. This enables hardware switching for filter/flows configured in software, and this is known as the fast path. From second packet onwards, all similar packets go through this fast path (or vSwitch acceleration path).

To support an exception path, CP_VSI is configured as the default VSI for the eSwitch for all packets received from the VFs. Packets from uplink are directed to the PF_VSI. Packets received on CP_VSI are directed to the corresponding VF_PR netdev based on the Source VSI in the RX descriptor. The PF_VSI is configured as the default VSI for uplink packets, and the frames received on PF_VSI are directed to UL_PR netdev.

Transmits from PR netdevs are treated as directed transmits. Transmits from UL_PR are directed to the network by setting the switch control tag to indicate uplink packet and bypass any hardware filters.

Exception path for VM-to-VM packets:

```
VF1_netdev → VF1_VSI → eSwitch → CP_VSI → VF1_PR netdev → OVS → VF2_PR netdev →  
CP_VSI → eSwitch → VF2_VSI → VF2_netdev
```

Exception path for uplink to VM packets:

```
PF_netdev → PF_VSI(H2) → eSwitch(H2) → Uplink(H2) → Uplink(H1) → eSwitch(H1) → PF_VSI(H1)  
→ UL_PR netdev → OVS → VF1_PR netdev → CP_VSI → eSwitch(H1) → VF1_VSI → VF1_netdev
```

Exception path for VM to uplink packets:

```
VF1_netdev → VF1_VSI → eSwitch → CP_VSI → UL_PR netdev → OVS → UL_PR netdev → PF_VSI →  
eSwitch(H1) → Uplink(H1) → Uplink(H2) → eSwitch(H2) → PF_VSI → PF_netdev
```

4.1.2.2 Bridge-Based Slow Data Path

Bridge-based slow data path is enabled by adding all the PR netdevs (VF_PR) to a Linux bridge. The bridge learns the MAC Addresses on the ports from the packets received via the exception path, and maintains a table of MAC Address-to-port combinations. This table is used to forward the packets to the right port based on the DMAC, or flood to all the other ports if the DMAC is a BUM (broadcast, multicast or unknown unicast) address. This is an example of classical L2 switching.

4.1.3 Technical Details: switchdev Mode and TC-Flower

TC-Flower enables implementing another data path in the Linux kernel using the TC subsystem through which the flow rules can be offloaded to hardware. The TC-Flower Classifier, along with TC actions infrastructure, is used to implement the TC data path. It can be configured by OVS or any control plane as a software implementation and/or a way to offload to hardware. The data path can be configured as one or more tables that can hold match/action flow rules.

Each flow rule can match on a set of well-known packet field's metadata and perform actions such as drop, modify, redirect, mirror, and so on.

Multiple tables are supported using TC chains, where each chain can be considered a table. The flow rules in each chain are organized per packet type (eth type). For each packet type, the rules are arranged in groups of flow rules, where each group contains rules with the same masks.

- Only one action per flow is supported (the last one in the list), though driver can parse action list.
- All TC Match fields have equal priority.

4.1.3.1 Switchdev Mode TC-Flower Hardware Offloads

In eSwitch switchdev mode, the device allows hardware offload of the L2/L3/L4 TC-Flower exact match rules via the PRs. TC-Flower can be used to offload the kernel data path.

The following rules are supported for hardware offload:

- L2
 - protocol (ip, ip6, 802.1q, 802.1ad)
 - dst_mac/src_mac
 - vlan_id
- L3
 - source_ip/destination_ip
- L4
 - Source-port/destination-port

The following actions are supported for HW offload:

- Drop (FLOW_ACTION_DROP)
- Redirect to an if index (FLOW_ACTION_REDIRECT)

Notes: The VLAN ID that is set from the PR netdev is always the outer VLAN for the VLAN. This can be of type 0x8100 (802.1q) or 0x88A8 (802.1ad).

If match/action is requested that cannot be offloaded because it is either unsupported or there are no resources, software must fail the offload and then flush all existing fast-path rules.

The following actions are unsupported in the 800 Series:

- VLAN push (id, prio)
- VLAN pop
- VXLAN encap/decap

5.0 eSwitch Configuration

This section explains how to change the eSwitch mode of a PF to switchdev, and further configuration like namespace, OVS-TC Filter, and steps for other features.

5.1 eSwitch Mode Configuration Between Legacy/switchdev

5.1.1 Overview

Consider the following important notes:

- Intel® Ethernet 800 Series Network Adapters support configuration of switchdev mode independently per physical port. This means that some ports can be in eSwitch Legacy mode, while others can be configured in eSwitch switchdev mode.
- The eSwitch mode (switchdev or legacy) is changeable without a reboot. Mode change commands must go into effect without reboot or driver reload.
- While in switchdev mode, the following configurations are not supported:
 - ADQ on the PF or VF
 - Link Aggregation on the PF
 - L2 forwarding on the PF
 - Trusted VFs (and “ip link commands” to configure a VF as trusted VF)
- RDMA and SR-IOV are both supported in switchdev mode, but RDMA must be enabled with a specific sequence:
 1. Change eSwitch mode to switchdev.
 2. Create SR-IOV VFs.
 3. Enable RDMA (load iRDMA driver).

Similarly, when removing the interface from the bridge or MAC/VLAN when RDMA is active, you must follow this exact sequence of steps:

1. Remove RDMA if it is active (rmmod iRDMA driver).
2. Destroy SR-IOV VFs if they exist.
3. Remove the interface from the bridge or MAC/VLAN.
4. Reactivate RDMA and recreate SR-IOV VFs as needed.

5.1.2 Variable Definitions

The following variables are used in this discussion and in the sample scripts in [Appendix A](#).

\$PF1	The Physical Interface (LAN port). eSwitch mode of this port is set as switchdev. This is also referred to as Uplink Port.
\$PF1_PCI	Used as <code>pci/0000:xx:xx.x</code> , where <code>0000:xx:xx.x</code> is PCI address of \$PF1 .
\$PF1_IP	IP Address assigned to \$PF1 .
\$BR	The Linux or OVS software bridge.
\$VF1 \$VF2	Two SR-IOV VFs associated with \$PF1 .
\$VF1_PCI \$VF2_PCI	PCI addresses for \$VF1 and \$VF2 associated with \$PF1 .
\$VF1_MAC \$VF2_MAC	MAC Addresses assigned to \$VF1 and \$VF2 .
\$VF1_IP \$VF2_IP	IP Addresses assigned to \$VF1 and \$VF2 .
\$VF1_PR \$VF2_PR	Port Representors for \$VF1 and \$VF2 .
\$MASK	Subnet Mask for IP Address for \$PF1 and \$VF1-PR , \$VF2-PR .

5.1.3 Configuration Steps

1. Verify that hardware, software, and firmware requirements are met, as detailed in [Section 3.0](#).
2. Remove all VFs from the PF under test. The 800 Series Network Adapter allows switching in and out of switchdev mode only if there are no VFs created/associated with related PF.
 - a. Stop all VMs, containers, or DPDK applications using VFs connected to the PF.
 - b. Unload all VFs from the PF by setting the number of VFs to 0:

```
echo 0 > /sys/class/net/$<PF1>/device/sriov_numvfs
```

Note: When the PF driver is already in switchdev mode, for each VF that is attached to the PF there is a corresponding VF_PR netdev. When the VF is removed, the corresponding VF_PR netdev is automatically removed as well.

3. Create the software vSwitch (Linux Bridge or OVS) and add the PF interface as an uplink to the vSwitch.

Note: This must be done before the PF is set to switchdev mode.

Bridge example:

```
ip link add $<BR> type bridge
ip link set $<PF1> master $<BR>
```

OVS example:

```
ovs-vsctl add-br $<BR>
ovs-vsctl add-port $<BR> $<PF1>
ovs-vsctl show
```

4. Use the Linux devlink API to change the eSwitch mode of the PF PCI device to switchdev or legacy mode.

Here, PF1_PCI is `pci/0000:xx:xx.x` which is the address of PCI device **\$PF1**. It can be found by:

```
lspci -D | grep $<PF1>
or
ethtool -i $<PF1>
devlink dev eswitch set $<PF1_PCI> mode switchdev
or
devlink dev eswitch set $<PF1_PCI> mode legacy
```

5. Check that the current eSwitch mode is changed.

```
devlink dev eswitch show $<PF1_PCI>
```

6. Create SR-IOV VFs after changing eSwitch mode.

```
echo <num_of_VFs> > /sys/class/net/$<PF1>/device/sriov_numvfs
```

Note: By default, the 800 Series driver always starts with all interfaces in Legacy mode. eSwitch switchdev settings and related filters persist with PF interface resets, but not driver reloads or system reboots. To persist switchdev mode settings between reboots, create a script to apply the changes at boot time.

5.2 Limitations and Troubleshooting

- When changing mode to/from switchdev on the PF or applying configurations to the VF, if there is any limitation, check the *dmesg* log for informational and warning messages with the PCI address of that PF/VF.
- Adding a physical port to a Linux Bridge will fail and result in "Device or resource busy" message if SR-IOV is already enabled on a given port.
- If any VFs are already bound to a PF and you try to change eSwitch mode for that PF, the device rejects the devlink command with the error "Operation not supported" and logs an informational message stating "DMESG: ice <pci/0000:xx:xx.x>: Changing eSwitch mode is allowed only if there is no VFs created".
- If ADQ is enabled on the PF that has eSwitch mode set as switchdev, the device rejects the devlink command with the error "Operation not supported" and log an informational message stating "DMESG: ice <pci/0000:xx:xx.x>: TC MQPRIO offload not supported, switchdev is enabled".
- eSwitch switchdev mode does not support trusted VFs and rejects the command with the error "Operation not supported".
- Enabling L2 forwarding is not supported with switchdev mode. If you try to enable L2 offload on port that is in switchdev mode, the command should be rejected with message "Could not change any device features" and log an information message as "DMESG: ice <pci/0000:xx:xx.x>: MACVLAN offload cannot be configured - switchdev is enabled."
- Switch filter must be maintained if the PF is reset (PFR). In case of a Global Reset (GLOBR) or Embedded Management Processor Reset (EMPR), the device must be able to re-learn the switch filters again through the slow path learning mechanism, and offload to hardware.
- Hardware offloaded Flow table supports the maximum number of rules (32K entry) and maximum number of recipes (64). The total number of offloaded flows in hardware depends on the number of tuple's field used in the flow.

- Default OVS max-idle (aging time) is 10 seconds. This means that after 10 seconds, all the hardware offloaded flows are deleted followed by slow path learning and again offloading to hardware. As a workaround, it is recommended to configure the max-idle value to a larger value (such as 10 hours) as follows:

```
ovs-vsctl set Open_vSwitch . other_config:max-idle=36000000
```

- After configuring SR-IOV VFs in switchdev mode, to confirm that all interfaces (PF and VFs) are connected to same switch instance, read the unique `phys_switch_id` as follows:

```
cat /sys/class/net/$(PF1)/phys_switch_id
```

or

```
cat /sys/class/net/$(VF_PR)/phys_switch_id
```

The `phys_switch_id` entry must show same value for all interfaces belonging to the same switchdev instance.

- When eSwitch mode is set as switchdev then host admin can read interface port name within the Ethernet Controller for associated interfaces by using PF or VF_PR as below -

```
cat /sys/class/net/$(PF1)/phys_port_name
```

or

```
cat /sys/class/net/$(VF_PR)/phys_port_name
```

5.3 TC-Flower Rule Hardware Offload Configuration

TC-Flower makes use of the Linux flow dissector to extract packet data into a flow key. The populated flow key is masked and matched against the rules of the classifier. If a match is found, actions associated with the matching rule are executed.

The following steps outline how to configure hardware offloaded TC-Flower rules:

1. Enable hardware offload on the interface.

When a TC-Flower rule is added, the flower classifier determines the offload device, if any, of the flow. The rule is offloaded to hardware if `NETIF_F_HW_TC` feature is supported and enabled on the PF and VF interfaces (PF and VF_PR). The hardware offload feature must be enabled via **ethtool** on the PF and VF (VF_PR).

```
ethtool -K $(PF1) hw-tc-offload on
```

or

```
ethtool -K $(VF_PR) hw-tc-offload on
```

2. TC-Flower hardware/software offload Flag.

Use `skip_sw` in the TC-Flower rule creation to enable hardware offload for the rule. Hardware offload is also controlled on a per-rule basis by using the flags `skip_hw` and `skip_sw`. These flags are mutually exclusive.

- `skip_hw` denotes that the rule is added to software but not hardware. An error is reported if the flow cannot be added to software. Do not process filter by hardware.
- `skip_sw` denotes that the rule is added to hardware but not software. An error is reported if the flow cannot be added to hardware. Do not process filter by software. If hardware has no offload support for this filter, or TC offload is not enabled for the interface, the operation fails.

- The default behavior is to try to add the rule to both hardware and software. No error is returned if the flow cannot be added to hardware, but an error is reported if the flow cannot be added to software. *skip_hw* does not use the fast path, so performance is limited.

The following example uses the *skip_sw* parameter to **tc** to add the rule to hardware but not software.

```
# tc qdisc add dev eth0 ingress
# tc filter add dev eth0 ingress \
protocol ip \
flower skip_sw \
ip_proto tcp dst_port 1234 \
action drop
```

3. If using OVS: Enable *hw_tc_offload* and hardware/software offload flag.

TC offload must be enabled for *skip_sw*. The following commands show how to enable *hw_tc_offload* and how to set hardware/software offload policy. Make sure to follow the sequence. First you must enable *hw_tc_offload*, then set the TC offload flag for *skip_hw* or *skip_sw*.

```
ovs-vsctl set Open_vSwitch. other_config:hw-offload=true
ovs-vsctl set Open_vSwitch. other_config:tc-policy=skip_sw
```

4. Verify offloaded flow in hardware.

The **tc** command-line tool makes use of two TCA CLS flags (*TCA_CLS_FLAGS_IN_HW* and *TCA_CLS_FLAGS_NOT_IN_HW*) to allow the user to control and inspect the placement of rules in hardware and software. These flags allow the kernel to report the presence of a rule in hardware.

- Check with the **tc -s monitor** command. The *in_hw* field indicates that the flow is present in hardware.

```
# tc filter show dev eth0 ingress
filter protocol ip pref 49152 flower chain 0 handle 0x1
eth_type ipv4
ip_proto tcp
dst_port 1234
skip_sw
in_hw
```

- With OVS, execute OVS flow dump:

```
ovs-appctl dpctl/dump-flows -m
```

In O/P, if the flag shows as *offloaded:yes*, *dp:tc*, it means flows are on hardware.

Table 1. Supported TC Match-Action fields

TC-Flower Match Field	TC Flow Dissector ID	Meaning	Notes
dst_mac src_mac	FLOW_DISSECTOR_KEY_ETH_ADDRS	Source and/or destination MAC Address	For L2 header.
vlan_id	FLOW_DISSECTOR_KEY_VLAN_ID	VLAN ID	For VLAN tag.
dst_ip src_ip	FLOW_DISSECTOR_KEY_IPV4_ADDRS	IPv4 source or destination IP Address	For address type IPv4.
dst_ip src_ip	FLOW_DISSECTOR_KEY_IPV6_ADDRS	IPv6 source or destination IP Address	For address type IPv6.
dst_port src_port	FLOW_DISSECTOR_KEY_PORTS	L4 source or destination port	For TCP/UDP port.

Appendix A Sample Scripts

Following are two sample scripts that explain how to change eSwitch mode to switchdev, create VFs, and configure OVS-TC filters. **It is recommended to follow proper sequence as detailed below.**

These scripts are to be used as examples only and must be modified for each environment and use case.

A.1 Script A: eSwitch switchdev Mode with Linux Bridge Configuration

The following commands are used to create and bring up two VFs in switchdev mode, and to configure TC-Flower PF filters. Namespaces on the host allow for easy testing of the switchdev feature without VM creation, but a similar exercise could be done with VMs instead of namespaces.

It can be used as a reference to run at boot time so PF eSwitch will boot in switchdev mode after every reboot.

```
=====

#!/bin/bash
set -x
#set -e

DEVLINK=devlink
TC=tc
$BR=br0
PF1=ens4f0      # (PF whose eSwitch will be configured in switchdev mode. Change accordingly.)
PF1_PCI="pci/0000:af:00.0"
PF1_IP=192.168.66.16
VF1=ens4f0v0
VF2=ens4f0v1
VF1_PCI=0000:af:01.0
VF2_PCI=0000:af:01.1
VF1_MAC=52:54:00:00:16:01
VF2_MAC=52:54:00:00:16:02
VF1_IP=192.168.66.161
VF2_IP=192.168.66.162
VF1_PR=eth0
VF2_PR=eth1
PEER_IP=192.168.66.10
MASK=24
PEER_MAC=68:05:ca:a3:7b:10

rmmod ice
modprobe ice
sleep 2

#1. Make sure that there are no VFs
echo 0 > /sys/class/net/$PF1/device/sriov_numvfs

#2. Create a bridge
ip link add $BR type bridge 2> /dev/null

# To allow PF to be added to bridge as uplink
# PF needs to be added to bridge prior to entering switchdev and creating VFs
#3. Add PF as UpLink port to the bridge
ip link set $PF1 master $BR

#4. Change eSwitch mode to switchdev
$DEVLINK dev eswitch set $PF1_PCI mode switchdev

# Check the current eSwitch mode
$DEVLINK dev eswitch show $PF1_PCI
```



```
#5. Create 2 SR-IOV VFs
echo 2 > /sys/class/net/$PF1/device/sriov_numvfs

#6. Configure VF MAC Addresses
ip link set $PF1 vf 0 mac $VF1_MAC
ip link set $PF1 vf 1 mac $VF2_MAC

#7. Add VF Port Representors to the bridge and bring all of them up
ip link set $VF1_PR master $BR
ip link set $VF2_PR master $BR
ip link set $VF1_PR up
ip link set $VF2_PR up
ip link set $PF1 up
ip link set $BR up

#8. Delete IP address on PF and assign IP address to bridge
ip addr del $PF1_IP/24 dev $PF1
ip addr add $PF1_IP/24 dev $BR

#9. Create 2 network namespaces: ns1, ns2
ip netns add ns1 2> /dev/null
ip netns add ns2 2> /dev/null
sleep 2

#10. Move VF1 and VF2 to ns
ip link set $VF1 netns ns1
ip link set $VF2 netns ns2

#11. Add IP Addresses and bring up VF interfaces moved to namespaces
ip netns exec ns1 ip link set $VF1 up
ip netns exec ns2 ip link set $VF2 up
ip netns exec ns1 ip addr add $VF1_IP/$MASK dev $VF1
ip netns exec ns2 ip addr add $VF2_IP/$MASK dev $VF2

# Enable hw-tc-offload on PF (Uplink port) and VF Port Representors
#12. To offload tc filters to the hardware hw-tc-offload must be enabled on the VFs Port
Representor (VF_PR)
ethtool -K $PF1 hw-tc-offload on
ethtool -K $VF1_PR hw-tc-offload on
ethtool -K $VF2_PR hw-tc-offload on

# Verify settings:
ethtool -k $PF1 | grep "hw-tc"
ethtool -k $VF1_PR | grep "hw-tc"
ethtool -k $VF2_PR | grep "hw-tc"

#13. Enable ingress qdisc on PF (Uplink port) and VF Port Representors
$TC qdisc add dev $PF1 ingress
$TC qdisc add dev $VF1_PR ingress
$TC qdisc add dev $VF2_PR ingress

#14. Add filter with skip_sw to offload to hardware

#Add tc filter for VF1 -> PEER (unicast ip)
$TC filter add dev $VF1_PR ingress protocol ip prio 1 flower src_mac $VF1_MAC dst_mac
$PEER_MAC skip_sw action mirred egress redirect dev $PF1

#Add tc filter for VF1 -> VF2 (unicast ip)
$TC filter add dev $VF1_PR ingress protocol ip prio 1 flower src_mac $VF1_MAC dst_mac
$VF2_MAC skip_sw action mirred egress redirect dev $VF2_PR

#Add tc filter for VF2 -> PEER (unicast ip)
$TC filter add dev $VF2_PR ingress protocol ip prio 1 flower src_mac $VF2_MAC dst_mac
$PEER_MAC skip_sw action mirred egress redirect dev $PF1

#Add tc filter for VF2 -> VF1 (unicast ip)
$TC filter add dev $VF2_PR ingress protocol ip prio 1 flower src_mac $VF2_MAC dst_mac
$VF1_MAC skip_sw action mirred egress redirect dev $VF1_PR
```

```
#Add tc filter for PEER -> VF1 (unicast ip)
$TC filter add dev $PF1 ingress protocol ip prio 1 flower src_mac $PEER_MAC dst_mac
$VF1_MAC skip_sw action mirred egress redirect dev $VF1_PR

#Add tc filter for PEER -> VF2 (unicast ip)
$TC filter add dev $PF1 ingress protocol ip prio 1 flower src_mac $PEER_MAC dst_mac
$VF2_MAC skip_sw action mirred egress redirect dev $VF2_PR

sleep 2

#15. Do a ping from VF1 to PEER
ip netns exec ns1 ping -c3 $PEER_IP

#16. Do a ping from VF2 to PEER
ip netns exec ns2 ping -c3 $PEER_IP

#17. Do a ping from VF1 to VF2
ip netns exec ns1 ping -c3 $VF2_IP
```

A.2 Script B: Switchdev Mode and OVS Configuration

Following are example step-by-step commands to create and bring up two VFs when the PF is in switchdev mode, and to configure data path through OVS. How to check OVS flows, offloaded fields, and filters is also included. Namespaces on the host allow for easy testing of the switchdev feature without VM creation, but a similar exercise could be done with VMs instead of namespaces.

```
#!/bin/bash
set -x
#set -e

DEVLINK=devlink
TC=tc
$BR=brl
PF1=ens4f0
PF1_PCI="pci/0000:af:00.0"
PF1_IP=192.168.66.16
VF1=ens4f0v0
VF2=ens4f0v1
VF1_PCI=0000:af:01.0
VF2_PCI=0000:af:01.1
VF1_MAC=52:54:00:00:16:01
VF2_MAC=52:54:00:00:16:02
VF1_IP=192.168.66.161
VF2_IP=192.168.66.162
VF1_PR=eth0
VF2_PR=eth1
PEER_IP=192.168.66.10
MASK=24
PEER_MAC=68:05:ca:a3:7b:10

#1. Load the ICE driver
rmmod ice
modprobe ice
sleep 2

#2. Install the Open vSwitch package & start the service
#2.1. Install the OVS package
zypper install openvswitch

#2.2. Start the open vSwitch service
systemctl status openvswitch
```

```
#2.3. Check the vSwitch status
systemctl status openvswitch

#3. Create an OVS bridge
ovs-vsctl add-br $BR

#4. Add PF as an Uplink Port to the bridge:
ovs-vsctl add-port $BR $PF1
ovs-vsctl show

#5. Check and change the Mode to switchdev
#eSwitch mode could be changed only if there are no VFs created and the PF has been added to the
OVS bridge
#PF1_PCI should look like that: pci/0000:03:00.1
#To Find it lspci -D | grep Eth

#5.1. Show current eSwitch mode - should be legacy
devlink dev eswitch show $PF1_PCI

#5.2. Change eSwitch mode to switchdev
devlink dev eswitch set $PF1_PCI mode switchdev

#5.3. Show current eSwitch mode - should be switchdev
devlink dev eswitch show $PF1_PCI
sleep 2

#6. Enable SRIOV and create 2 VFs
echo 2 > /sys/class/net/$PF1/device/sriov_numvfs
sleep 2

#7. Enable hw-tc-offload on PF (Uplink port) and VF Port Representors
ethtool -K $PF1 hw-tc-offload on
ethtool -K $VF1_PR hw-tc-offload on
ethtool -K $VF2_PR hw-tc-offload on

#8. Configure OVS (Enable hardware offload, which is disabled by default)
ovs-vsctl set Open_vSwitch . other_config:hw-offload=true

# tc flow placement. one of: none, skip_sw, skip_hw
ovs-vsctl set Open_vSwitch . other_config:tc-policy=skip_sw

#9. Restart Open Switch service
systemctl restart openvswitch

#10. Add VF Port Representors to OVS bridge
ovs-vsctl add-port $BR $VF1_PR
ovs-vsctl add-port $BR $VF2_PR

#Set them to UP State
ip link set $PF1 up
ip link set $VF1_PR up
ip link set $VF2_PR up

#11. Configure VFs
#create 1 network namespace for each VF
ip netns add ns1 2> /dev/null
ip netns add ns2 2> /dev/null
sleep 1

#12. Move VFs to namespaces
ip link set $VF1 netns ns1
ip link set $VF2 netns ns2

#13) Set VFs to up state and give them IP Addresses
ip netns exec ns1 ip link set $VF1 up
ip netns exec ns2 ip link set $VF2 up
ip netns exec ns1 ip a $VF1_IP/24 dev $VF1
ip netns exec ns2 ip a $VF2_IP/24 dev $VF2
```

```
#14. Enable bridge
ip link set $BR up

#15. Check connections and Watch rules being added via tc tool
#ping 2nd VF from 1st VF
ip netns exec ns1 ping -c3 $VF2_IP

#16. Watch rules being added via tc tool
tc -s monitor
# If Flag shows as in_hw that means flows are offloaded to hardware

#17. Check connections and Watch rules offloaded via OVS
ip netns exec ns1 ping -c3 $VF2_IP
ovs-appctl dpctl/dump-flows -m

# In output, check for offload and datapath field as offloaded: yes, dp:tc, it means flows are
offloaded to HW.

=====
```

Appendix B Glossary and Acronyms

Table 2. Definition of Terms

Term	Definition
CP_VSI	Control Plane Virtual Station Interface
EMPR	Embedded Management Processor Reset
eSwitch	Ethernet controller's embedded switch. Sometimes referred to as Virtual Ethernet Bridge (VEB).
FDB	Forwarding Database
GLOBR	Global Reset
MTU	Maximum Transmission Unit
OVS	Open Virtual Switch OVS. Sometimes referred to as vSwitch.
PF	Physical Function
PFR	Physical Function Reset
PF_PR	Physical Function Port Representor
PF_VSI	Physical Function Virtual Station Interface
PR	Port Representor
RSS	Receive Side Scaling
UL_PR	Uplink Port Representor
VEB	Virtual Ethernet Bridge. Sometimes referred to as eSwitch.
VF	Virtual Function
VF_PR	Virtual Function Port Representor
VLAN	Virtual Local Area Network
VSI	Virtual Station Interface
vSwitch	Virtual switch. Sometimes referred to as Open Virtual Switch (OVS).



LEGAL

No license (express or implied, by estoppel or otherwise) to any intellectual property rights is granted by this document.

This document (and any related software) is Intel copyrighted material, and your use is governed by the express license under which it is provided to you. Unless the license provides otherwise, you may not use, modify, copy, publish, distribute, disclose or transmit this document (and related materials) without Intel's prior written permission. This document (and related materials) is provided as is, with no express or implied warranties, other than those that are expressly stated in the license.

Intel disclaims all express and implied warranties, including without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement, as well as any warranty arising from course of performance, course of dealing, or usage in trade.

This document contains information on products, services and/or processes in development. All information provided here is subject to change without notice. Contact your Intel representative to obtain the latest forecast, schedule, specifications and roadmaps.

The products and services described may contain defects or errors which may cause deviations from published specifications.

Copies of documents which have an order number and are referenced in this document may be obtained by calling 1-800-548-4725 or by visiting www.intel.com/design/literature.htm.

Intel and the Intel logo are trademarks of Intel Corporation in the U.S. and/or other countries.

Other names and brands may be claimed as the property of others.

© 2021 Intel Corporation.